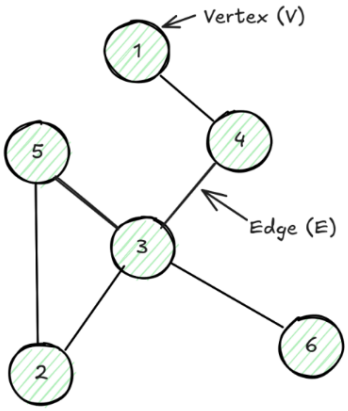
GRAPH

No 8

Struktur Data

Apa Itu Graph

Graph adalah struktur data yang digunakan untuk menggambarkan hubungan antara pasangan objek. Graph terdiri dari simpul (juga disebut verteks) dan sisi yang menghubungkan simpul-simpul ini. Graph banyak digunakan dalam berbagai bidang, seperti ilmu komputer, biologi, ilmu sosial, dan banyak lagi, untuk memodelkan hubungan dan interaksi. Berikut adalah ilustrasi dan komponen-komponen dasar sebuah graph.



Berdasarkan gambar yang tertera dapat diambil informasi bahwa notasi yang digunakan pada struktur data Graph merupakan G = (V, E) di mana:

- V (Vertex): Lingkaran-lingkaran yang diberi label angka (1, 2, 3, 4, 5, 6) merepresentasikan simpul-simpul atau node-node dari graph. Simpul bisa berupa objek, entitas, atau lokasi, dan merupakan komponen fundamental yang dihubungkan oleh sisi.
- E (Edges): Garis-garis yang menghubungkan simpul-simpul tersebut merepresentasikan sisi-sisi. Setiap sisi menghubungkan dua simpul, menunjukkan adanya hubungan di antara mereka. Sebagai contoh, sisi antara simpul 1 dan simpul 3 menunjukkan adanya koneksi antara kedua simpul ini. Pada graph tersebut, Vertices dan Edge terdiri dari:
- Vertex: {1, 2, 3, 4, 5, 6}
- Edges: Graph ini mencakup sisi-sisi seperti (1, 4), (2, 5), (3, 4), (3, 2), (3, 5), dan (3, 6).

Komponen Pada Graph

Untuk membuat sebuah graph, terdapat beberapa komponen yang harus dimiliki oleh

graph tersebut, yakni:

1. Simpul (Node): Unit dasar dari sebuah graph, yang merepresentasikan objek atau entitas. Setiap simpul dapat berisi data atau hanya mewakili sebuah pengenal.
2. Sisi (Edges): Koneksi antara simpul-simpul. Sebuah sisi bisa berarah atau tidak berarah.
   - Sisi Berarah: Menunjukkan hubungan satu arah (dari simpul A ke simpul B).
   - Sisi Tidak Berarah: Menunjukkan hubungan dua arah (antara simpul A dan simpul B).
3. ***Opsional** Bobot (costs) : Edge dapat memiliki bobot (atau biaya) yang terkait dengannya, yang merepresentasikan jarak, waktu, atau ukuran lain dari koneksi antara node.

Istilah Pada Graph

Di dalam graph terdapat beberapa istilah, yakni:

1. Incident

   Jika terdapat busur e yang menghubungkan simpul-simpul v dan w (ditulis e = (v, w)), maka v dan w disebut "terletak" pada e, dan e dikatakan incident dengan v dan w.

2. Degree (Derajat), Indegree, dan Outdegree

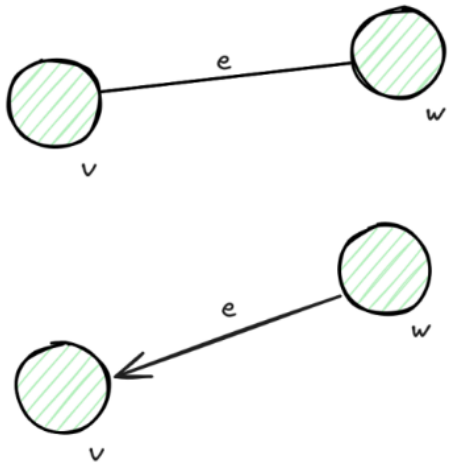
   Degree dari sebuah simpul adalah jumlah busur yang berhubungan langsung(incident) dengan simpul tersebut.

   - Indegree pada graph berarah adalah jumlah busur yang "masuk" ke simpul,atau jumlah busur yang ujungnya incident dengan simpul tersebut.

- Outdegree adalah jumlah busur yang "keluar" dari simpul, atau jumlah busur yang pangkalnya incident dengan simpul tersebut.

3. Adjacent

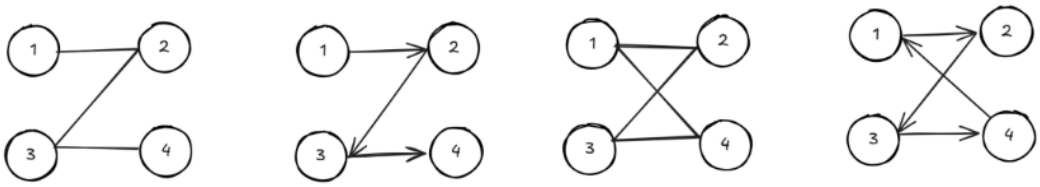
   Pada graph tak berarah, dua simpul dikatakan adjacent jika ada busur yang menghubungkan keduanya. Pada graph berarah, simpul (node) v dikatakan bersebelahan (adjacent) dengan simpul w jika terdapat busur (edge) dari v ke w, seperti yang digambarkan di bawah.



4. Successor dan Predecessor

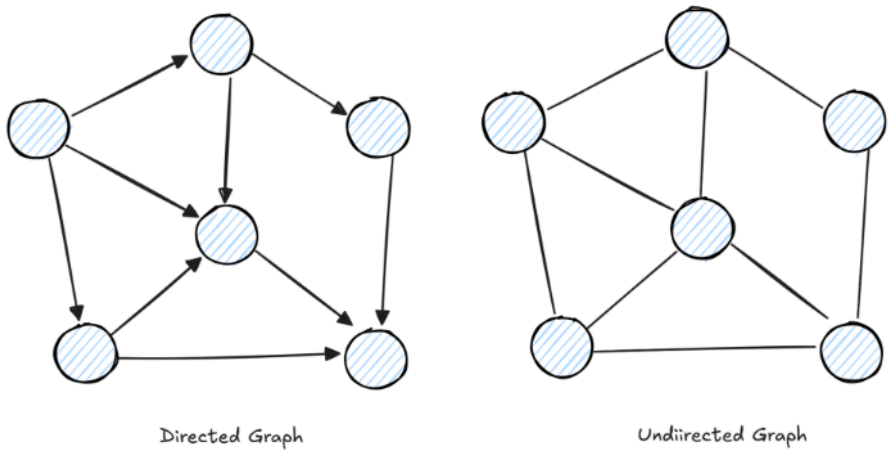
   Dalam graph berarah, jika simpul v adjacent dengan simpul w, maka v disebut successor dari w, dan w disebut predecessor dari v.

5. Path Path adalah serangkaian simpul yang berbeda yang saling adjacent secara berturut-turut, dari satu simpul ke simpul berikutnya.



Graph Berarah dan Tidak Berarah

Graph dapat dikategorikan menjadi graph berarah dan graph tak berarah berdasarkan sifat dari sisi-sisi yang menghubungkan simpul-simpulnya. Perhatikan perbedaan keduanya pada gambar di bawah ini.



Dalam graph tak berarah, sisi-sisi yang menghubungkan simpul-simpul tidak memiliki arah. Artinya, jika ada sisi antara simpul A dan simpul B, kita bisa bergerak dari A ke B maupun dari B ke A. Hubungan antara simpul-simpul tersebut bersifat timbal balik atau dua arah. Sehingga memiliki karakteristik sebagai berikut:

- Hubungan Simetris: Jika A terhubung dengan B, maka B juga terhubung dengan A.
- Notasi Sisi: Untuk sisi antara simpul A dan B, dinyatakan sebagai (A, B). Yang sama dengan (B, A).

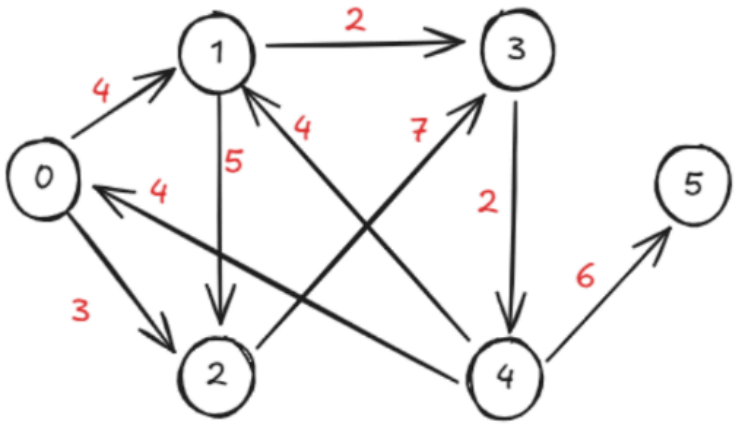
Sedangkan dalam graph berarah, setiap sisi memiliki arah. Artinya, jika ada sisi dari simpul A ke simpul B, kita bisa bergerak dari A ke B, namun belum tentu dari B ke A (kecuali ada sisi lain yang secara eksplisit menghubungkan B ke A). Hubungan antar simpul bersifat satu arah atau berarah. Sehingga memiliki karakteristik:

- Hubungan Asimetris: Sisi dari A ke B tidak berarti ada sisi dari B ke A.

- Notasi Sisi: Sisi dari simpul A ke simpul B dinyatakan sebagai A → B. Yang menyatakan hubungan satu arah dari A ke B.

Graph Berbobot

Sebuah graph dikatakan memiliki bobot jika setiap busur di dalamnya memiliki nilai yang mencerminkan hubungan antara dua simpul. Bobot ini berfungsi untuk menunjukkan beragam atribut dari hubungan tersebut. Sebagai contoh, dalam sebuah graph yang merepresentasikan jaringan jalan, bobot pada busur dapat melambangkan panjang jalan antar dua titik (simpul) tertentu, atau bahkan rata-rata volume kendaraan yang melintasi jalan tersebut setiap hari. Oleh karena itu, graph berbobot sangat efektif untuk memodelkan berbagai jenis hubungan yang melibatkan ukuran atau nilai spesifik, seperti jarak, waktu, hingga frekuensi interaksi. Gambar di bawah ini menunjukkan contoh graph berbobot, di mana setiap garis (busur) memiliki nilai numerik yang merepresentasikan bobotnya.

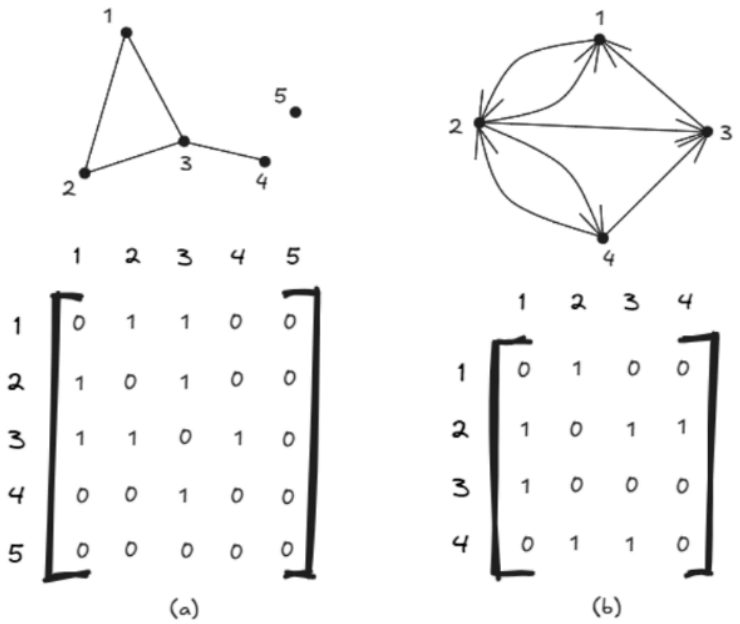


Weighted Graph

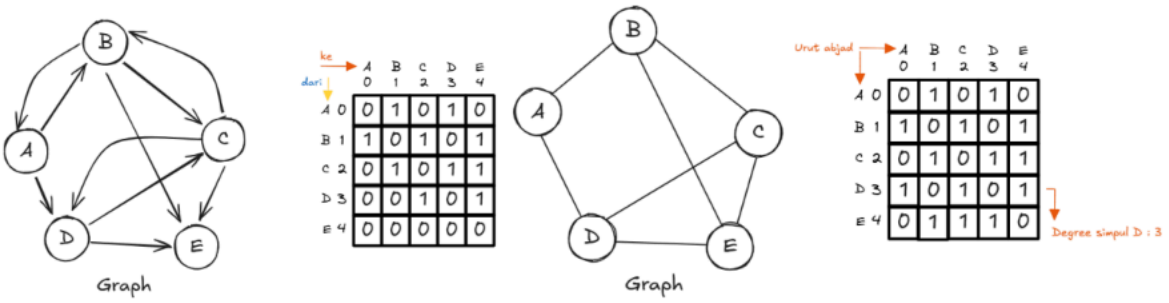
Panjang busur (atau bobot) mungkin tidak selalu digambarkan secara proporsional dengan bobotnya. Sebagai contoh, sebuah bobot 5 bisa saja digambarkan lebih panjang dibandingkan dengan bobot 7.

Representasi Graph

Graph, baik yang berarah maupun yang tidak berarah dapat direpresentasikan dalam sebuah matriks maupun linked list, sebagai contoh dapat dilihat pada gambar berikut.



Derajat (Degree) dari setiap node bisa dihitung dengan menjumlahkan angka satu di setiap barisnya. Misalnya, pada gambar (a), node 1 memiliki total 2 karena ada dua angka 1 yang bisa dijumlahkan dalam matriks. Jadi, kita bisa katakan bahwa node 1 memiliki derajat 2. Contoh lain terdapat pada gambar berikut:



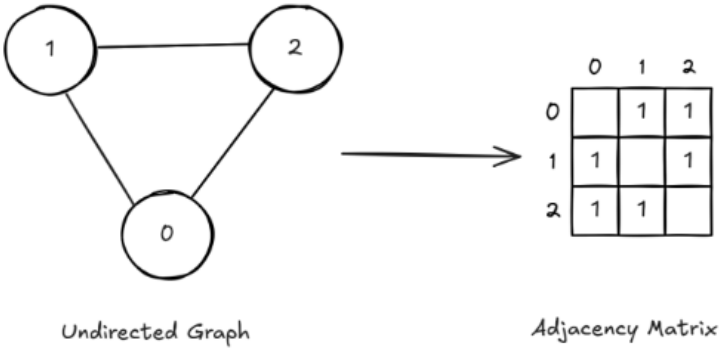
Representasi Adjacency Matrix

Adjacency matrix adalah sebuah cara untuk merepresentasikan sebuah graph sebagai sebuah matriks boolean (berisi angka 0 dan 1). Mari kita asumsikan terdapat n verteks di dalam graph. Maka, kita akan membuat sebuah matriks 2D adjMat[n][n] yang memiliki dimensi n x n.

- Jika terdapat sebuah edge dari vertex i ke j, maka adjMat[i][j] diberi nilai 1.
- Jika tidak terdapat edge dari vertex i ke j, maka adjMat[i][j] diberi nilai 0.

Representasi Undirected Graph sebagai Adjacency Matrix

Gambar di bawah ini menunjukkan sebuah undirected graph. Pada awalnya, seluruh isi Matrix diinisialisasi dengan nilai 0. Jika terdapat sebuah edge dari source ke destination, kita akan mengisi nilai 1 pada kedua posisi (adjMat[source][destination] dan adjMat[destination][source]), karena jalurnya bersifat dua arah.



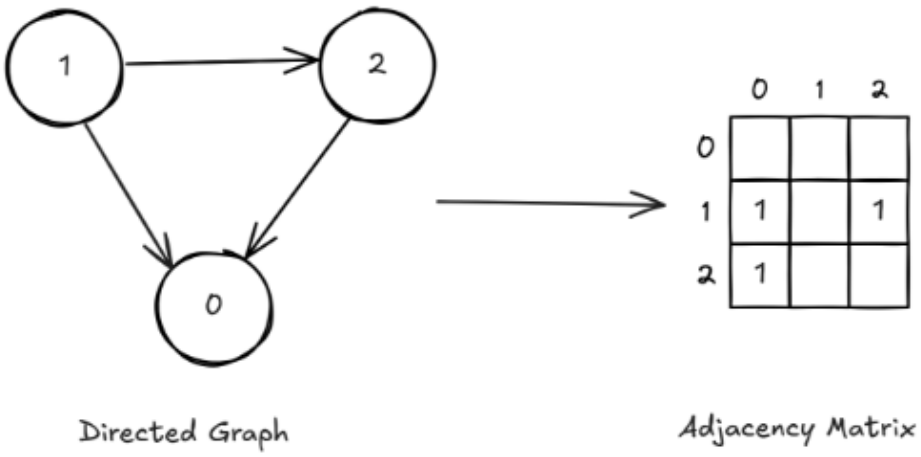
Biar makin paham soal Representasi Undirected Graph sebagai Matriks Adjacency, kamu bisa langsung coba salin dan jalankan kode C++ di bawah ini.

```
// Program C + buat nunjukkin Adjacency Matrix
// untuk graph yang undirected (tak berarah) dan unweighted (tak
// berbobot)
#include <iostream>
#include <vector>
using namespace std;
```

```
void addEdge(vector<vector<int >> &mat, int i, int j) {
    mat[i][j] = 1;
    mat[j][i] = 1; //Karena graph-nya undirected, jadi berlaku dua arah
}
void displayMatrix(vector<vector<int >> &mat) {
    int V = mat.size();
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++)
            cout << mat[i][j] << " ";
    cout << endl;
    }
}
int main() {
// Bikin graph isinya 4 vertex, awalnya belum ada edge
// Ingat, semua nilainya otomatis diisi 0 dulu
    int V = 4;
    vector<vector<int >> mat(V, vector<int>(V, 0));

//Sekarang kita tambahin edge-nya satu-satu

    addEdge(mat, 0, 1);
    addEdge(mat, 0, 2);
    addEdge(mat, 1, 2);
    addEdge(mat, 2, 3);
    cout << "Adjacency Matrix Representation" << endl;
    displayMatrix(mat);
return 0;
}
```

Representasi Directed Graph sebagai Adjacency Matrix

Gambar di bawah ini menunjukkan sebuah directed graph. Pada awalnya, seluruh isi Matrix diinisialisasi dengan nilai 0. Jika terdapat sebuah edge dari source ke destination, kita hanya mengisi nilai 1 pada posisi spesifik adjMat[source][destination] tersebut, karena jalurnya bersifat satu arah.



Representasi Adjacency List

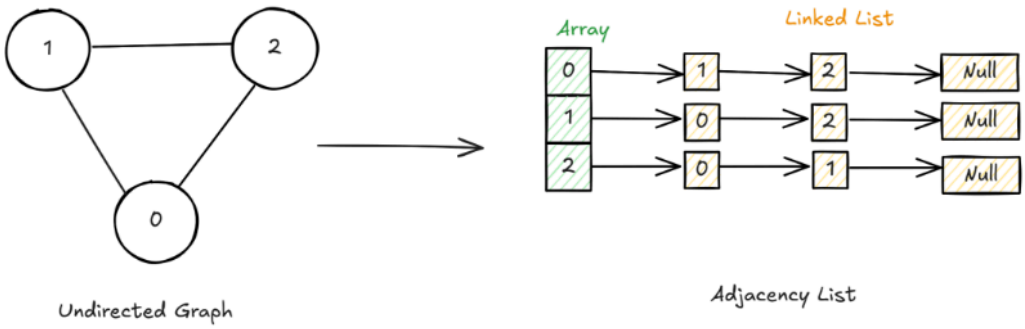
Sebuah array dari List digunakan untuk menyimpan edge yang menghubungkan dua vertex. Ukuran array ini sama dengan jumlah vertex (misalnya n). Setiap index di dalam array ini mewakili sebuah vertex spesifik di dalam graph. Entri pada index i dari array tersebut berisi sebuah linked list yang menyimpan vertex-vertex yang bertetangga (adjacent) dengan vertex i. Jadi, jika kita asumsikan ada n vertex di dalam graph, kita akan

membuat sebuah array dari list berukuran n dengan nama adjList[n].

- adjList[0] akan berisi semua node yang terhubung (bertetangga) dengan vertex 0.
- adjList[1] akan berisi semua node yang terhubung (bertetangga) dengan vertex 1,dan begitu seterusnya.

Representasi Undirected Graph sebagai Adjacency List

Undirected graph di bawah ini memiliki 3 vertex. Oleh karena itu, kita akan membuat sebuah array of list dengan ukuran 3, di mana setiap index mewakili masing-masing vertex. Proses pengisiannya seperti ini:



Biar makin paham soal Representasi Undirected Graph sebagai Adjacency List, kamu bisa langsung coba salin dan jalankan kode C++ di bawah ini.

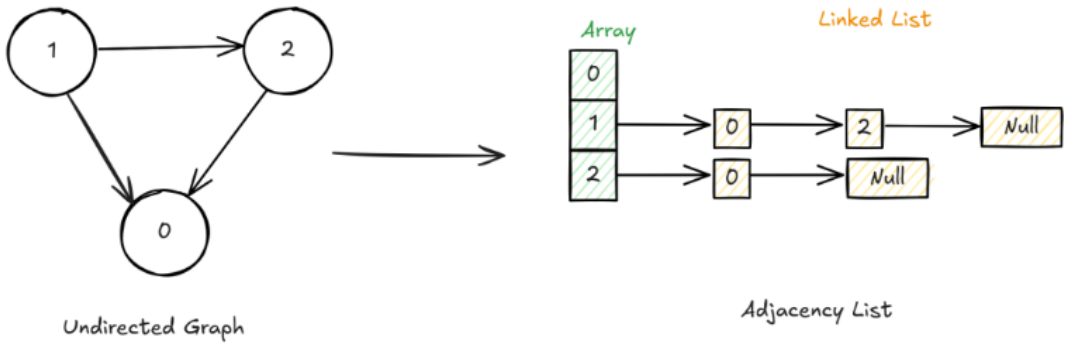
```
#include <iostream>
#include <vector>
using namespace std;
// Fungsi buat nambahin edge (sambungan) antar dua vertex
void addEdge(vector<vector<int >> &adj, int i, int j) {
    adj[i].push_back(j);
    adj[j].push_back(i); // Dibikin dua arah, soalnya graph-nya undirected
}
// Fungsi buat nampilin adjacency list-nya
void displayAdjList(const vector<vector<int >> &adj) {

    for (int i = 0; i < adj.size(); i++) {
        cout << i << ": "; // Cetak nomor vertex-nya
        for (int j : adj[i]) {
            cout << j << " "; // Tampilin tetangga-tetangganya
    }
cout << endl;
    }
}
// Program utamanya
```

```
int main() {
// Bikin graph dengan 4 vertex, awalnya masih kosong (belum ada edge)
    int V = 4;
    vector<vector<int >> adj(V);

// Terus kita tambahin edge-nya satu per satu
    addEdge(adj, 0, 1);
    addEdge(adj, 0, 2);
    addEdge(adj, 1, 2);
    addEdge(adj, 2, 3);

    cout << "Adjacency List Representation:" << endl;

    displayAdjList(adj);
    return 0;
}
```

Representasi Directed Graph sebagai Adjacency List

Directed graph di bawah ini memiliki 3 vertex. Maka, kita akan membuat sebuah array of list berukuran 3, di mana setiap index mewakili masing-masing vertex. Proses pengisiannya seperti ini:

- Pertama, kita lihat vertex 0. Karena tidak ada edge yang keluar darinya, maka list pada index 0 akan kosong.
- Selanjutnya, vertex 1 memiliki dua tetangga (neighbours), yaitu vertex 0 dan 2. Jadi, kita masukkan vertex 0 dan 2 ke dalam list pada index 1.
- Terakhir, lakukan hal yang sama untuk vertex 2. Masukkan semua vertex yang menjadi tujuannya ke dalam list pada index yang sesuai.



Graph Traversal

Graph traversal adalah proses mengunjungi setiap vertex (simpul) dan edge (sisi) dalam Graph secara sistematis. Terdapat dua algoritma utama untuk melakukan ini: Breadth-FirstSearch (BFS) dan Depth-First Search (DFS).

Breadth-First Search (BFS)

Breadth-First Search (BFS) adalah algoritma penjelajahan yang mengunjungi vertex secara "melebar". Artinya, algoritma ini akan mengunjungi semua vertex tetangga dari vertex awal terlebih dahulu, sebelum melanjutkan ke vertex di level berikutnya. Untuk melacak urutan vertex yang akan dikunjungi, BFS menggunakan struktur data Queue (antrean). Bayangkan Kamu berada di sebuah pesta dan ingin menyapa semua orang.

- Kamu mulai dari temanmu (titik awal).
- Kemudian, Kamu menyapa semua teman dari temanmu tersebut (tetangga level 1).
- Setelah itu, barulah Anda menyapa semua teman dari teman-teman yang baru saja Kamu sapa (tetangga level 2), dan begitu seterusnya.

Pendekatan ini memastikan Anda mengunjungi orang-orang terdekat terlebih dahulu sebelum bergerak lebih jauh. Inilah inti dari BFS. Langkah-langkah Algoritma BFS:

1. Pilih sebuah vertex sebagai titik awal.
2. Masukkan vertex awal ke dalam Queue.
3. Tandai vertex awal sebagai "sudah dikunjungi".
4. Selama Queue tidak kosong, lakukan: a. Keluarkan satu vertex dari depan Queue (sebut saja u). b. Kunjungi semua vertex tetangga (v) dari u yang belum pernah dikunjungi. c. Untuk setiap tetangga v tersebut: i. Tandai v sebagai "sudah dikunjungi". ii. Masukkan v ke dalam Queue.

Implementasi BFS dengan C++

Berikut adalah source code C++ untuk mengimplementasikan algoritma BFS.

```
#include <iostream>
#include <list>
#include <queue>
#include <vector>


using namespace std;
// Fungsi untuk menambahkan edge pada graph
// Menerima adjacency list by reference (&) agar bisa diubah
void addEdge(vector<list<int >> &adj, int v, int w) {
    adj[v].push_back(w); // Tambahkan w ke list milik v
}


// Fungsi untuk menjalankan BFS
void BFS(vector<list<int >> &adj, int V, int s) {
    // Membuat array boolean untuk menandai vertex yang sudah dikunjungi
    vector<bool> visited(V, false);

    // Membuat queue untuk BFS
    queue<int> q;

    // Tandai vertex awal sebagai sudah dikunjungi dan masukkan ke queue
    visited[s] = true;
    q.push(s);

    cout << "Hasil penjelajahan BFS dimulai dari vertex " << s << " :" << endl;


    while (!q.empty()) {
        // Keluarkan vertex dari depan queue dan cetak
        s = q.front();
        cout << s << " ";
        q.pop();
        // Dapatkan semua tetangga dari vertex s yang baru dikeluarkan
        // Jika ada tetangga yang belum dikunjungi, tandai dan masukkan ke
queue
        for (auto const &i : adj[s]) {
```

```
            if (!visited[i]) {
                visited[i] = true;
                q.push(i);
            }
        }
    }
cout << endl;
}

int main() {
    // Tentukan jumlah vertex
    int V = 6;

    // Membuat adjacency list untuk merepresentasikan graph
    // Ukurannya disesuaikan dengan jumlah vertex (V)
    vector<list<int >> adj(V);


    // Menambahkan edge ke graph dengan memanggil fungsi addEdge
    addEdge(adj, 0, 1);
    addEdge(adj, 0, 2);
    addEdge(adj, 1, 3);
    addEdge(adj, 2, 4);
    addEdge(adj, 2, 5);
    addEdge(adj, 3, 5);
/*
Struktur Graph yang kita buat:
         0
        / \
         1 2
        / / \
        3 4 5
        \ /
        (edge 3 >5)

*/

    // Menjalankan BFS dimulai dari vertex 0
    BFS(adj, V, 0);
    // Menjalankan BFS dimulai dari vertex 2
    BFS(adj, V, 2);
    return 0;}
```

Depth-First Search (DFS)

Depth-First Search (DFS) adalah algoritma penjelajahan yang bekerja dengan cara menelusuri satu cabang dari sebuah graph sejauh mungkin (sedalam mungkin) sebelum akhirnya mundur (backtracking) untuk menelusuri cabang lain. Berbeda dengan BFS yang menggunakan Queue, algoritma DFS menggunakan struktur data Stack. Seringkali, stack ini diimplementasikan secara implisit melalui pemanggilan fungsi rekursif. Bayangkan Anda sedang berada di dalam sebuah labirin.

- Anda memilih satu lorong (cabang) dan terus menyusurinya sampai menemukan jalan buntu atau jalan keluar.
- Jika Anda menemukan jalan buntu, Anda akan mundur (backtrack) ke persimpangan terakhir dan mencoba lorong lain yang belum pernah Anda lewati.
- Anda mengulangi proses ini sampai seluruh area labirin telah dijelajahi.

Cara kerja ini adalah inti dari DFS, yaitu "dalam dulu, baru melebar".

Langkah-langkah Algoritma DFS (Menggunakan Rekursi):

1. Pilih sebuah vertex sebagai titik awal.
2. Tandai vertex tersebut sebagai "sudah dikunjungi" dan cetak nilainya.
3. Untuk setiap vertex tetangga yang belum pernah dikunjungi, panggil kembali fungsi DFS untuk vertex tetangga tersebut (ini adalah langkah rekursifnya).

Proses ini akan terus berlanjut hingga semua vertex yang terhubung dari titik awal telah dikunjungi.

Implementasi DFS dengan C++

Berikut adalah source code C++ untuk mengimplementasikan algoritma DFS.

```
#include <iostream>
#include <list>
#include <vector>
```

```
using namespace std;

// Fungsi untuk menambahkan edge pada graph (sama seperti sebelumnya)
void addEdge(vector<list<int >> &adj, int v, int w) {
    adj[v].push_back(w); }

// Fungsi pembantu (helper) rekursif untuk DFS
void DFS_util(vector<list<int > &adj, int v, vector<bool> &visited) {
    // Tandai vertex saat ini sebagai sudah dikunjungi dan cetak
    visited[v] = true;
    cout << v << " ";

    // Lakukan rekursi untuk semua tetangga vertex ini
    for (auto const &i : adj[v]) {
        if (!visited[i]) {
        DFS_util(adj, i, visited);
        }
    }
}


// Fungsi utama DFS yang memanggil fungsi rekursif
void DFS(vector<list<int >> &adj, int V, int s) {
    // Membuat array boolean untuk menandai vertex yang sudah dikunjungi
    // Awalnya semua ditandai false
    vector<bool> visited(V, false);

    cout << "Hasil penjelajahan DFS dimulai dari vertex " << s << " : ";

    // Memanggil fungsi utilitas rekursif untuk memulai penjelajahan
    DFS_util(adj, s, visited);

    cout << endl;
}

int main() {
    int V = 6;
    vector<list<int >> adj(V);

    // Menggunakan graph yang sama dengan contoh BFS
    addEdge(adj, 0, 1);
    addEdge(adj, 0, 2);
```

```
    addEdge(adj, 1, 3);
    addEdge(adj, 2, 4);
    addEdge(adj, 2, 5);
    addEdge(adj, 3, 5);
/*
Struktur Graph yang kita buat:
     0
    / \
    1  2
    / / \
    3 4 5
    \ /
    (edge 3 >5)

*/


    // Menjalankan DFS dimulai dari vertex 0
    DFS(adj, V, 0);
    // Menjalankan DFS dimulai dari vertex 2
    DFS(adj, V, 2);
return 0;
}
```